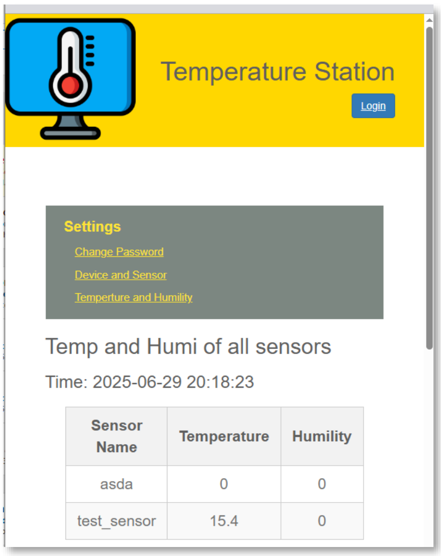
Temperature and Humility Station
Build Your Own

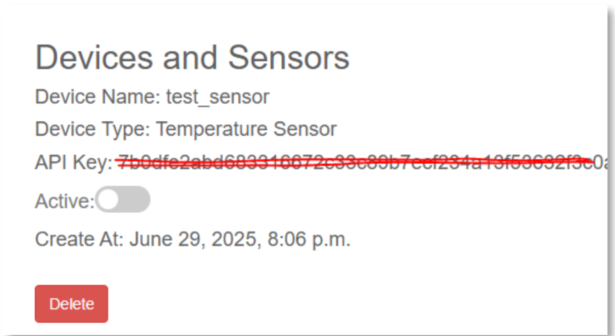
Contents

Introduction

There are 2 parts in this project. The first one is to build a LCD display clock using ESP32, 2x16 LCD module and temperature sensors. It will show the current date time and the sensor information. You can build as many as display clocks and put that round your building with different type of IoT sensors as you design.

Part 2 is to build a server that collect all the sensor data and present the latest sensor information such as temperature and humility on a webserver. The ESP32 sensors will send data every 2 mins using API calls. API Keys are stored on the server. User can disable / enable any key resulting allow or reject the data from individual sensor.





Hardware

ESP32-C3 super mini	Support 5v and 3.3v out, wifi , BT. It's perfect for this project
ESP32 CP2102 (Optional)	Support 5v and 3.3v out, wifi, BT and more GPIO pins
ESP32-S3 (not recommend)	Support 3.3v out only. It has the most GPIO pins
LCD1602 (Optional)	I2C 2 x 16 LCD display. 5V input
DHT 11 (your pick)	Indoor temperature and humility sensor / Cheap

	Input 3.3-5V / 0 – 50° C (± 2° C) / 20 – 90%RH (± 5%RH)
DHT 22 (your pick)	Temperature and humility sensor Input 3.3-5V / -40 – 80 ° C (± 0.5° C) / 0 – 100%RH (± 2%)
DS18B20 (your pick)	Outdoor waterproof temperature sensor Input 3.3-5V / -55 – 125 ° C
Raspberry Pi 3b	32Gb SD-card and Raspberry Lite OS

Build the ESP32 and Sensors

ESP32 Wiring - LCD and Temp Sensor

You will need update the main.py file with the correct pin numbers according to the sensor and the LCD display you are going to use.

Below are the default values in the main.py for ESP32 CP2102 module

TEMP_PIN = 2

SDA_PIN = 21

SCL_PIN = 22

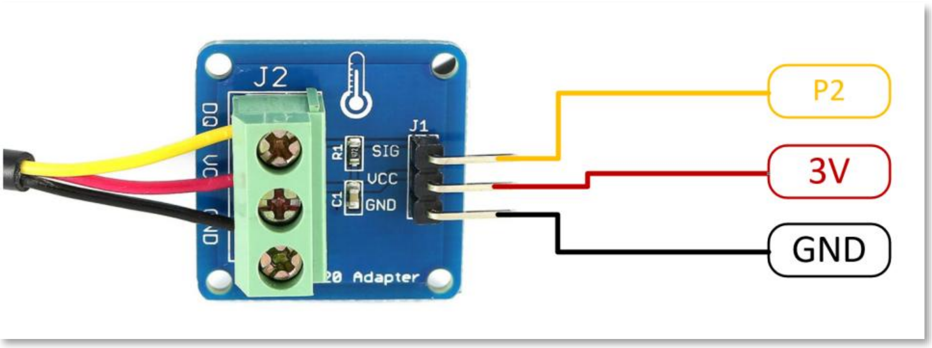
For ESP32-3C temperature SIG pin can go to GPIO 5. Display SDA → pin 8 and SCL → pin 9

Temperature Sensor Wiring (example)

DS18B20 can work on either 3v or 5v. I use 3v to reserve the 5v pin for LCD display

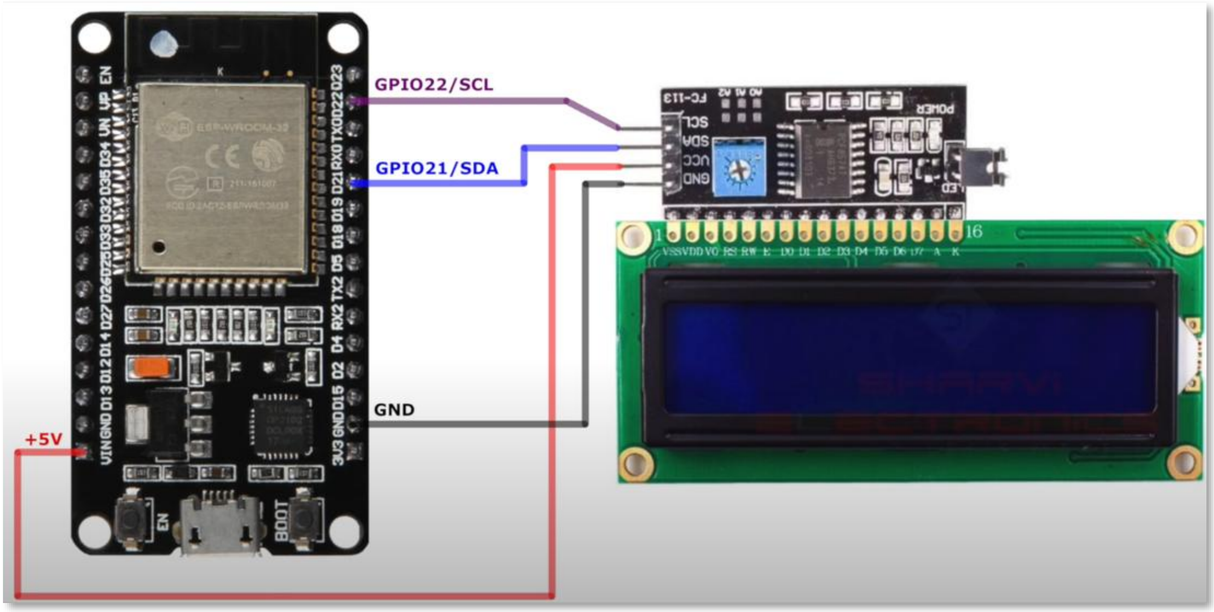
VCC <--> 3V, GND <--> Ground, SIG <--> GPIO 2

Same apply to sensor DHT11 and DHT22. SIG pin goes to GPIO pin 2 or any available GPIO accordingly. Please refer to the ESP32 wiring diagram.



LCD Wiring (example)

VCC <--> 5V, GND <-->Ground, SDA <-->GPIO21, SCL <-->GPIO22



Flash MircroPython firmware to ESP32

- Version: ESP32_GENERIC-20250415-v1.25.0.bin

  This version support https which required for API call or error with “ImportError: no module named 'ssl'

- Set ESP32 to download model

  hold down the BOOT button (often labeled as GPIO0) and then briefly press and release the EN (enable/reset) button

- Using windows with python installed to flash firmware

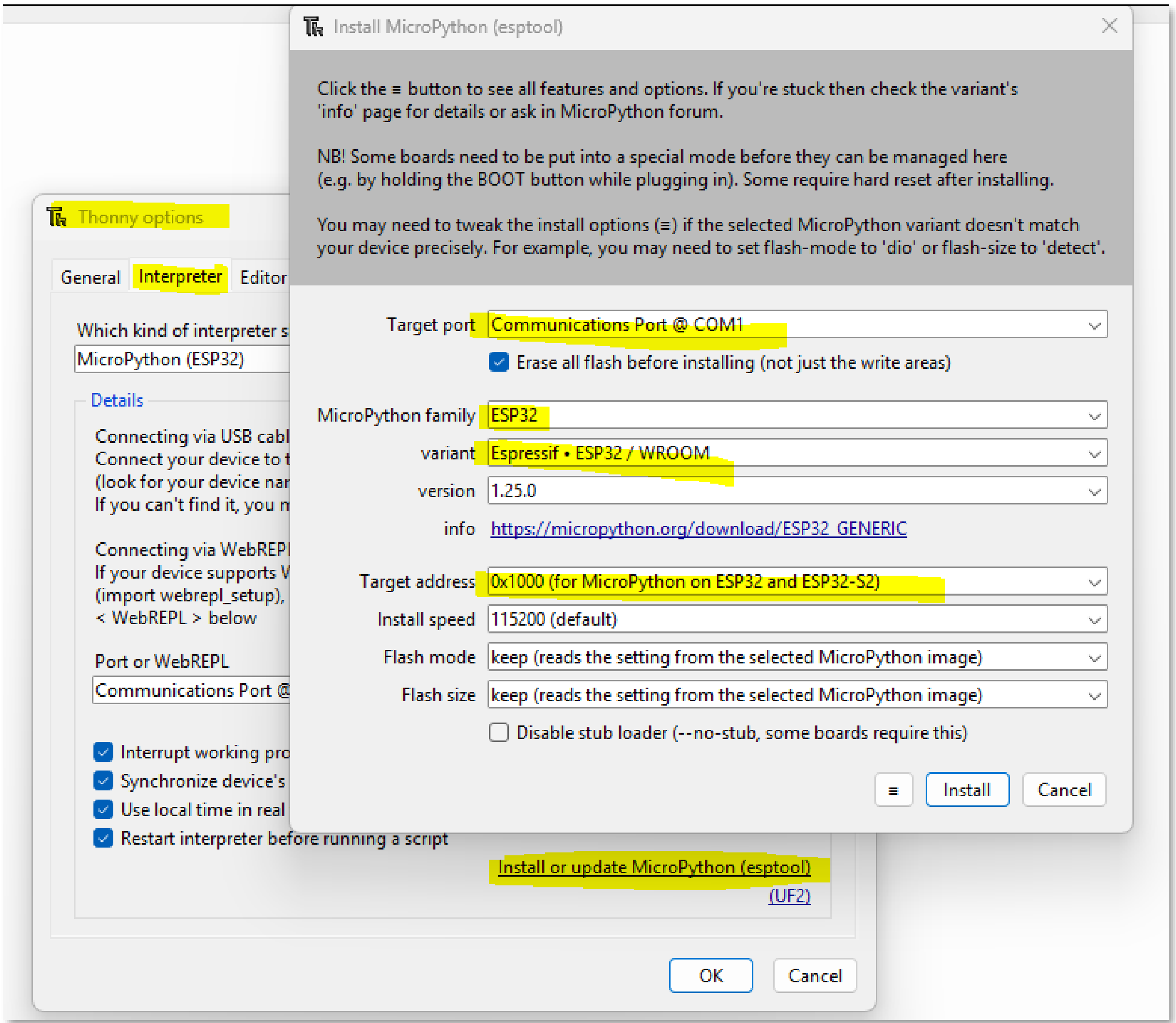
  Find the COM port number from Windows’ Device Manager. Below two python commands will erase and flash a generic Micro Python firmware to the ESP32. --chip esp32 or --chip esp32-c3 or --chip esp32-s3

  python -m esptool --chip esp32 --port COM3 erase_flash

  python -m esptool --chip esp32 --port COM3 --baud 460800 write_flash 0 C:\Path\To\ESP32_GENERIC-20250415-v1.25.0.bin

- Using Thonny application to flash firmware (Optional)

  Google it. It is easier. You will need Thonny to dump all python source files to ESP32 anyway.



Python code for ESP32

- Download and install Thonny for Windows (free Tools)
- Connect the ESP32 in Thonny

  Tools → Thonny Options → Interpreter → Select MicroPython (ES32) → Select the Communication Port @ COM?? → OK

- Upload below file to ESP32 including folder

  /lib/i2c_lcd.py

  /lib/lcd_api.py

  /lib/ntptime.py

  /boot.py

  /main.py

  /display_LCD1602.py

  /sensors.py

/local_time.py

/upload.py

/websetup.py

- Edit the main.py file to meet your need

  TEMP_PIN, SDA_PIN and SCL_PIN

  Day Light saving settings

  Timezone settings

Setup Django Web Server

We can Ubuntu server VM or Raspberry Pi OS with SSH enable with full access

- Ubuntu server 24.04.01 (minimal footprint install / 2 CPU / 2G RAM / 20Gb Disk)
  - If the system running on VM. Make sure all disk space has been used.

    sudo lvextend -l +100%FREE /dev/ubuntu-vg/ubuntu-lv

    sudo resize2fs /dev/mapper/ubuntu--vg-ubuntu—lv

```
user@filesafer:~$ df -h
Filesystem                         Size  Used Avail Use% Mounted on
tmpfs                              192M  976K  192M   1% /run
/dev/mapper/ubuntu--vg-ubuntu--lv   18G  4.0G   13G  24% /
tmpfs                              960M     0  960M   0% /dev/shm
tmpfs                              5.0M     0  5.0M   0% /run/lock
/dev/sda2                          1.8G   99M  1.6G   7% /boot
tmpfs                              192M   12K  192M   1% /run/user/1000
```

- Raspberry Pi 3 or 4B
  - Install Raspberry Pi OS Lite (no Desktop needed)

Install Django Framework and necessary Apps

sudo apt-get update

sudo apt-get install python3-django python3-pip python3-venv

- If you are using Ubuntu server 20.04 with minimal footprint installation

  sudo apt-get install nano cron iputils-ping

  sudo nano /etc/resolv.conf

  add line: **nameserver 8.8.8.8**

Create virtual environment and install Django

cd /

sudo mkdir Automation

sudo chmod 777 Automation

python3 -m venv Automation

cd Automation

source bin/activate

pip3 install Django

```
user@filesafer:~$ cd /
user@filesafer:/$ sudo mkdir Automation
user@filesafer:/$ sudo chmod 777 Automation/
user@filesafer:/$ python3 -m venv Automation
user@filesafer:/$ cd Automation
source bin/activate
pip3 install Django
Collecting Django
  Downloading django-5.2.2-py3-none-any.whl.metadata (4.1 kB)
Collecting asgiref>=3.8.1 (from Django)
  Downloading asgiref-3.8.1-py3-none-any.whl.metadata (9.3 kB)
Collecting sqlparse>=0.3.1 (from Django)
  Downloading sqlparse-0.5.3-py3-none-any.whl.metadata (3.9 kB)
Downloading django-5.2.2-py3-none-any.whl (8.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 8.3/8.3 MB 6.3 MB/s eta 0:00:00
Downloading asgiref-3.8.1-py3-none-any.whl (23 kB)
Downloading sqlparse-0.5.3-py3-none-any.whl (44 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 44.4/44.4 kB 3.8 MB/s eta 0:00:00
Installing collected packages: sqlparse, asgiref, Django
Successfully installed Django-5.2.2 asgiref-3.8.1 sqlparse-0.5.3
(Automation) user@filesafer:/Automation$
(Automation) user@filesafer:/Automation$
```

Create web applications and install dependent libraries

- Perform below commands under directory /Automation within the virtual environment

  django-admin startproject Thermometer .

  python3 manage.py startapp Login

  python3 manage.py startapp TempLog

  pip3 install ipcalc six yt_dlp django-sslserver

- Above process creates below folder structure

  Thermometer

  TempLog

  Login

Copy the source code files and setup

We will use sftp to cory the source files into created directories from above

If you are using windows computer you can use MoxaXterm free application to do this. Linux user can perform this task natively using Terminal.

- Start from the directory where you have downloaded the Django Source Code. SFTP to the server

    cd /Automation

    put -R *

    python3 manage.py makemigrations

    python3 manage.py migrate

- Create superuser for the Web App. This account is for you to login to the Web GUI

    python3 manage.py createsuperuser

```
(Automation) user@filesafer:/Automation$ python3 manage.py createsuperuser
Username (leave blank to use 'user'): user
Email address:
Password:
Password (again):
Superuser created successfully.
(Automation) user@filesafer:/Automation$
```

- Setup application auto-start (My ubuntu user account called “user”. It used in @reboot code below)

    sudo nano /etc/crontab

    - add below line to the bottom and save / exit

      @reboot user /bin/bash -c "/Automation/run_http.sh"

Setup and configure the web server

- Setup Nginx (Engine X) as front-end web server
    - Install Nginx

      sudo apt-get install nginx

    - Setup web site in nginx by creating file “Django” in /etc/nginx/sites-available/

      sudo nano /etc/nginx/sites-available/Django

        - Copy below to the file, save and exit

      server {

        listen 443 ssl;

        server_name <server FQDN or IP Address>;

        ssl_certificate /Automation/Cert/server.crt;

        ssl_certificate_key /Automation/Cert/server.key;

```
        location / {
            proxy_pass http://127.0.0.1:8000;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
        }
    }
    server {
        listen 80;
        server_name your.domain.or.ip;
        return 301 https://$host$request_uri;
    }
```

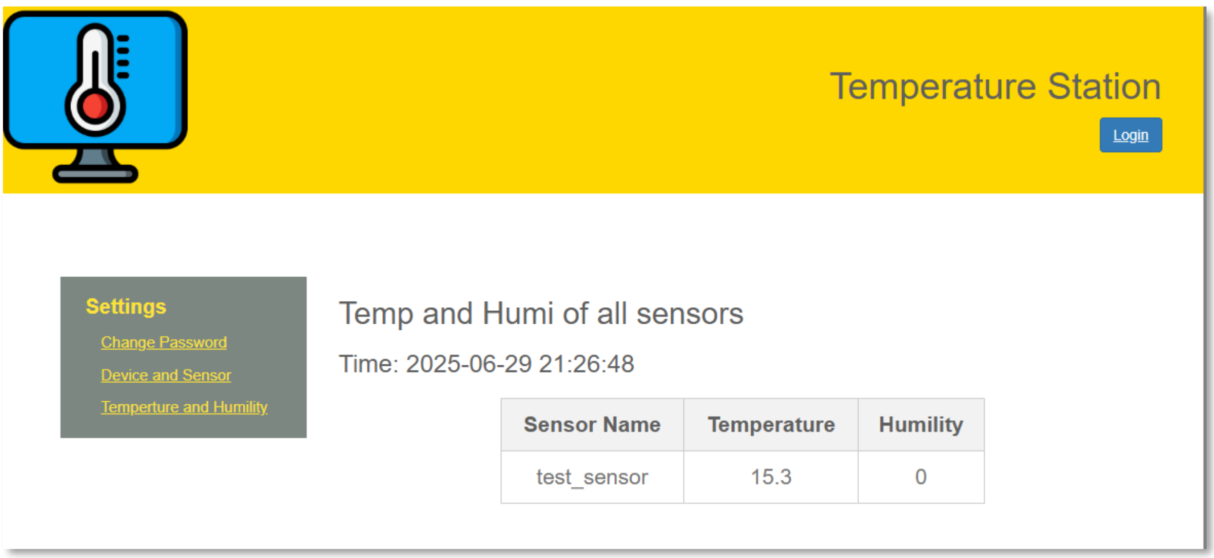
  - Start Nginx
    sudo ln -s /etc/nginx/sites-available/Django /etc/nginx/sites-enabled/
    sudo nginx -t
    sudo systemctl restart nginx

- Set correct timezone (optional) before Rebooting the server.

    sudo timedatectl set-timezone <local timezone such as Australia/Sydney>

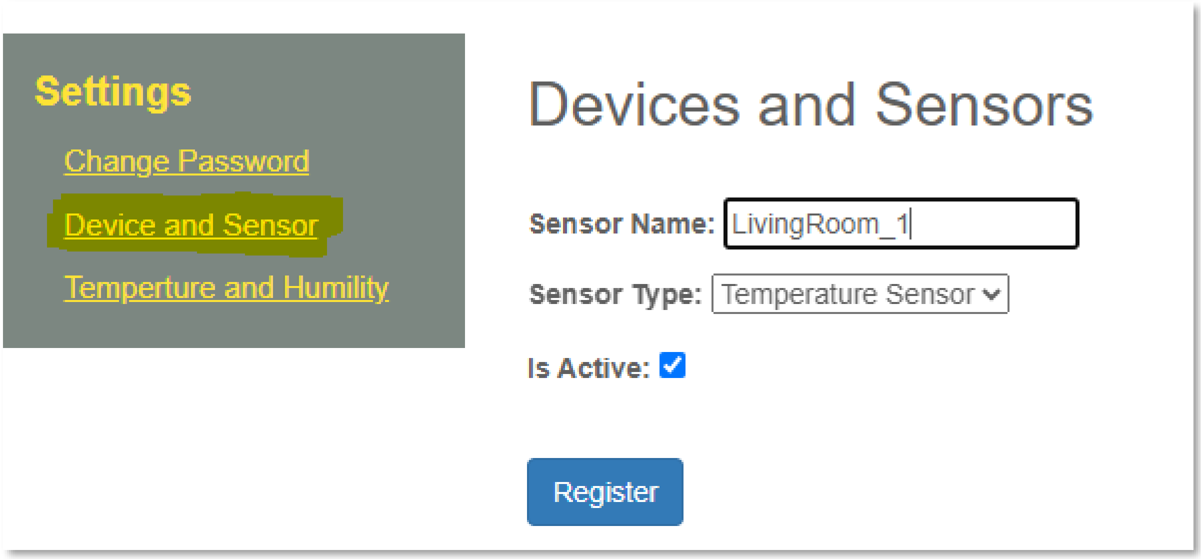
    sudo reboot now

  - **https://<your-server-ip-address>**



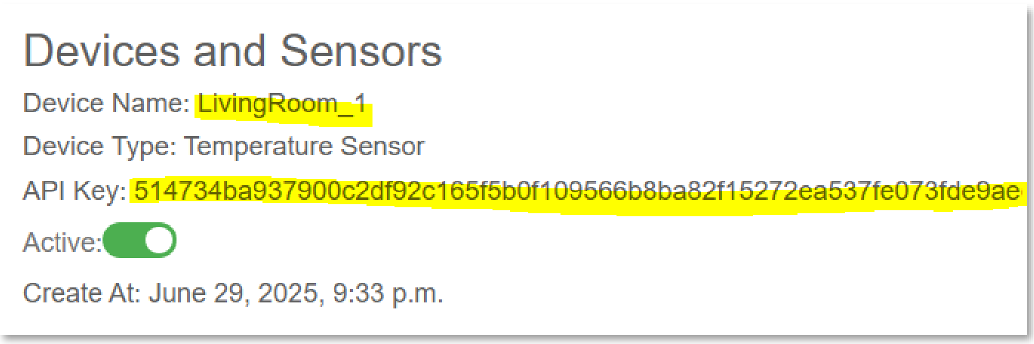
Register / create API key for sensor device

Device and Sensor → Register New Sensor → Put in a name for Device Name → Select the sensor type → Click Register



Setup ESP32 when it powers up at the first time

- Browse the server from your phone and copy the API key from website



- Search SSID “Set-me-up” and connect with password as password. Browse http://192.168.0.1
- Fill in all necessary detail then click “Save and Reboot”
  - Wifi name
  - Wifi password
  - Server IP
  - Device name (as above)
  - API Key (as above / paste)
- The ESP32 should reboot and connect to local wifi. You should see the display showing live information. The Website will should the temperature and humidity in 3 mins (average of last 3 mins)
- If the ESP32 can not connect to local wifi. The “set-me-up” network will show up again otherwise you will not see “Set-me-up” anymore.